

PvrGPU Shader Core & USC Subsystem — Block Diagram, Data Path & Pseudocode

本文件以 block diagram、data path flow 及 pseudocode 形式詳述 `src/systemc/shader/` (包含 `pco_decoder`、`usc_slot`、`usc_cluster` 與 `pco_iss`) 之 PowerVR USC (Unified Shading Cluster) 與 PCO 指令集模擬器 (ISS) 的架構與執行行為。

USC 是 PowerVR 的可程式化著色核心，本子系統負責將 Mesa 編譯產生的 PCO 機器碼解碼為語意指令、進行 4-lane Vertex / 2x2 Fragment Quad 槽位仲裁排程、在虛擬暫存器上忠實執行頂點/片段著色，並處理紋理採樣 (SMP) 暫停與接續 (Continuation/WDF) 機制。

0. Top-Level Architecture Block Diagram

USC Shader Subsystem — Architecture Overview

負責 Vertex 與 Fragment 兩階段之 PCO 解碼、排程與叢集執行
包含 PcoDecoder、UscSlot、UscCluster 及 Pcolss 模擬核心

↓ [Vertex Pipeline 流程] sc_fifo<PipelineTxn> (來自 VertexFetch / VertexPds)

① PcoDecoder (Vertex Stage)

IN: state.vertex_code (Mesa PCO binary bytes)
OUT: state.vertex_instructions, PcoProgramSummary
STAGE: kVertexPdsReady -> kVertexDecoded

↓ sc_fifo<PipelineTxn>

② UscSlot (Vertex Stage)

- 仲裁 Vertex 4-lane issue group 執行槽位
- 驗證 VertexLane 數量與分組計算
STAGE: kVertexDecoded -> kVertexIssued

↓ sc_fifo<PipelineTxn>

③ UscCluster (Vertex Stage)

- 執行 Vertex PCO ISS 指令 (矩陣運算、MVP Transform)
- 讀取 VTXIN 暫存器，寫入 VTXOUT0..63 (含 Clip Position)
- 輸出 emitted / ended_task 旗標
STAGE: kVertexIssued -> kVertexShaded (傳給 ClipCull)

↓ ----- [Fragment Pipeline 流程] sc_fifo<PipelineTxn> (來自 ParameterBuffer)

④ PcoDecoder (Fragment Stage)

IN: state.fragment_code (Mesa PCO binary bytes)
OUT: state.fragment_instructions, early_hsr_safe 標記
STAGE: kParameterBufferReady -> kFragmentDecoded

↓ sc_fifo<PipelineTxn> (經 TileScheduler/ISP/Frontend/PDS)

⑤ UscSlot (Fragment Stage)

- 仲裁 2x2 Quad 空間任務與 UscFragmentTask 槽位
- 驗證 Varying 系數庫 (Coefficient Banks) 與遮罩合法性
STAGE: kPdsReady -> kFragmentIssued

↓ sc_fifo<PipelineTxn>

⑥ UscCluster (Fragment Stage)

- 執行 Fragment PCO ISS (FITRP 內插、色彩運算、SMP 採樣)
 - 若遇到 SMP: 暫存狀態 (Continuation) 並向 TPU 發出請求
 - 接收 TPU 回應後透過 WDF 喚醒並寫入 PIXOUT0..3
- STAGE: kFragmentIssued -> kFragmentShaded (傳給 Texture/PBE)

1. Dual-Stage Data Path Flow (頂點與片段資料流)

[Vertex Data Path Flow]

1. PcoDecoder: 讀取 state.vertex_code -> 產出 PcoInstruction[]
2. VertexPds / UscSlot: 分配 4-lane Group (state.vertex_groups)
3. UscCluster:
R: state.vertex_lanes[].vertex_input (VTXIN0..31)
R: state.vertex_shared_registers (常數/Uniforms)
Execute: Pcolss (FADD, FMUL, FMAD, FSUB, MOV, etc.)
W: state.vertex_lanes[].vertex_output (VTXOUT0..63)
W: state.counters.vs_alu/tex/memory_instructions (展開累加)
傳送至 ClipCull (Stage: kVertexShaded)

[Fragment Data Path Flow]

1. PcoDecoder: 讀取 state.fragment_code -> 產出 PcoInstruction[]
2. FragmentFrontend / PDS: 組裝 2x2 FragmentQuads, CoefficientBanks
3. UscSlot: 驗證 UscFragmentTasks 與 Quad 遮罩
4. UscCluster:
R: state.fragment_invocations[] (像素 X/Y、深度、重心座標)
R: state.usc_coefficient_banks (1/W 及 Varyings 平面參數)
R: state.fragment_shared_registers (紋理描述符/常數)
Execute Path A (非紋理/平坦著色):
Pcolss 執行 FITRP 內插與色彩計算 -> 直接寫出 PIXOUT0..3
Execute Path B (含紋理採樣 SMP):
Pcolss 執行到 SMP 指令 -> 產生 TextureSampleRequest[]
暫存暫存器狀態到 PcoFragmentContinuation[]
發送至 TPU (Stage: kFragmentTexturePending)
TPU 採樣完成 -> 回傳 TextureSampleResponse[]
UscCluster::Resume: 載入 Continuation, 執行 WDF 喚醒
寫出最終 PIXOUT0..3 到 state.fragment_outputs[]
W: state.counters.fs_alu/tex/memory_instructions
傳送至 TextureUnit / PBE (Stage: kFragmentShaded)

2. PcoDecoder 內部處理流程

PcoDecoder — Internal Processing Flow

Step 1: 階段檢查與 Binary 載入

- Stage == Vertex: ASSERT kVertexPdsReady
Stage == Fragment: ASSERT kParameterBufferReady
LoadArray(state.vertex_code / state.fragment_code)

Step 2: 嚴格解碼 (DecodePcoProgram)

- 解析變長 Instruction Group (Phase 0, 1, 2)
驗證 Opcode、暫存器索引、Repeat Count、Flags
產出 PcoProgramSummary (Group/Instruction/I-O Masks)
計算靜態 ALU / Texture / Memory 指令分佈

Step 3: 介面契約驗證 (Contract Validation)

Vertex: 驗證 VTXIN 遮罩是否在 AttributeBindings 內
驗證 VTXOUT 遮罩與 Linkage (Clip Pos, Varying)
Fragment: 驗證 PIXOUT0..3 必須完整寫入
提取 early_hsr_safe 屬性

Step 4: 輸出與管線推進

state.vertex/fragment_instructions = store(decoded)
記錄 DrawList 靜態統計數據
cycles = decode_base + ceil(groups / batch)
WaitForCycles(cycles); output.write(txn)

3. UscSlot 槽位仲裁流程

UscSlot — Internal Processing Flow

Step 1: 著色階段檢查

Vertex Stage: ASSERT kVertexDecoded
Fragment Stage: ASSERT kPdsReady

Step 2: 工作單元分組驗證 (Task Group Validation)

Vertex:
groups = CeilDivide(vs_invocations, 4) (4-lane)
Fragment:
驗證 fragment_invocations[] 與 active 像素一致
驗證 fragment_quads[] 與 usc_fragment_tasks[] 匹配
驗證 coefficient_banks 大小與 Varying ABI 一致

Step 3: 推進至 Issued 階段

cycles = usc_slot_base + ceil(groups / slot_batch)
Stage -> kVertexIssued 或 kFragmentIssued
WaitForCycles(cycles); output.write(txn)

4. UscCluster 執行核心流程

UscCluster — Internal Processing Flow

[Vertex Mode]

- 載入 VertexLane[] 與 SharedRegisters
- FOR EACH lane IN lanes:
execution = ExecuteVertexPco(summary, insns,
lane.vtxin, ctx)
lane.vertex_output[0..63] = execution.outputs
lane.emitted = 1, lane.ended = 1
- 計算動態指令數: Repeat 展開 $\times$ vs_invocations
- Stage -> kVertexShaded; output.write(txn)

[Fragment Standard Mode (無紋理)]

1. 載入 FragmentInvocation[]
2. FOR EACH inv IN invocations:
ctx.sample_x = inv.x, ctx.sample_y = inv.y
ctx.coefficients = task.coefficients
execution = ExecuteFragmentPco(summary, insns, ctx)
outputs[i].pixel_output = execution.pixel_outputs
3. 計算動態指令數：Repeat 展開 × ps_invocations
4. Stage -> kFragmentShaded; output.write(txn)

5. 紋理採樣暫停與接續 (SMP & Continuation) 機制

當 Fragment Shader 包含紋理取樣 (SMP) 時，USC 採用硬體風格的非同步中斷/接續機制：

USC Cluster (Phase 1: 執行至 SMP 暫停)

1. 走訪 2×2 Quad 的 4 個 Lane (含 Helper Pixels)
2. 執行 PCO 指令 (如 FITRP 內插 UV 座標)
3. 遇到 SMP 指令：
 - 擷取 UV 座標與 Shared 暫存器內的 Texture/Sampler 描述符
 - 封裝為 TextureSampleRequest
 - 儲存當前暫存器與程式計數器至 PcoFragmentContinuation
 - 標記 execution.suspended = 1
4. 發送所有 Requests 至 TPU (texture_request_output)
5. PipelineStage 轉為 kFragmentTexturePending

↓ texture_request_output -> TPU

TPU (Texture Processing Unit)

1. 執行 Texel 讀取與 Bilinear / Trilinear 過濾
2. 生成 TextureSampleResponse[] (RGBA8 -> Float32)
3. 回傳至 USC (Stage: kTextureSamplesReady)

↓ texture_response_input -> USC

USC Cluster (Phase 2: 接續執行與 WDF 喚醒)

1. 讀取 TextureSampleResponse[] 與 PcoFragmentContinuation[]
2. 呼叫 ResumeFragmentPco():
 - 恢復 Temporaries 暫存器
 - 執行 WDF (Wait For Data Fence)，將 RGBA 載入目標暫存器
 - 繼續執行後續 PCO 指令 (如顏色調變、Alpha 運算)
 - 最終寫入 PIXOUT0..3 暫存器
3. 過濾掉 Helper Pixels，僅保留有效像素輸出至 FragmentOutput[]
4. PipelineStage 轉為 kFragmentShaded

6. PCO 暫存器檔案與指令集架構 (ISA)

6.1 暫存器檔案 (Register Banks)

Bank 代號	名稱	數量	說明
kVertexInput	VTXIN	32	Vertex Fetch 填入的頂點屬性原始位元 (IEEE-754 / 整數)
kVertexOutput	VTXOUT	64	頂點著色輸出 (0..3 為 Clip Position，其餘為 Varyings)

kPixelOutput	PIXOUT	4	片段著色色彩輸出 (RGBA 32-bit Float Bits)
kTemporary	TEMP	32	著色器執行過程中的暫存暫存器 (r0..r31)
kShared	SH	32	PDS/Driver 傳入的常數、Uniforms 或紋理描述符
kCoefficient	COEFF	132	PDS 載入的 24.8/32-bit 平面插值係數 (A/B/C/PAD)

6.2 核心指令群 (PcoOpcode 語意)

Opcode	類別	功能說明
kFloatAdd, kFloatSubtract	ALU	32-bit 浮點加減法 (FADD / FSUB)
kFloatMultiply, kFloatMad	ALU	32-bit 浮點乘法 (FMUL) 與乘加 (FMAD: $\alpha*b + c$)
kFloatMin, kFloatMax	ALU	浮點極值選取 (FMIN / FMAX)
kReciprocal, kReciprocalSquareRoot	ALU	快速倒數 ($1/x$) 與平方根倒數 ($1/\sqrt{x}$)
kFloatLog2, kFloatExp2	ALU	基底為 2 之對數與指數運算
kFloatFloor, kFloatGreaterEqual	ALU	地板函數與比較運算 (FGE)
kConditionalSelect	ALU	條件選取指令 (CONDSEL)
kMoveBypass, kMoveImmediate	Data	暫存器搬移與常數立即值寫入 (MOV / MOVI)
kFloatInterpolatePerspective	Interp	FITRP 空間平面透視插值運算 (計算 Varying)
kTextureSample, kTextureSampleLod	Texture	SMP / SMPLOD 紋理採樣請求
kWaitDataFence	Sync	WDF 資料屏障等待 (等待紋理或記憶體回傳)
kUvswrite, kUvswriteEmitEndTask	Export	UVSW 頂點輸出寫入與任務結束發射
kBranch, kBranchConditional	Flow	分支與條件分支跳躍
kDiscard	Flow	片段捨棄指令 (Discard)

7. 關鍵資料結構與 Handle Map

7.1 Shader 階段 MemoryPool Handles

Handle 名稱	儲存資料型別	生產者	主要消費者
vertex_code	uint8_t[]	Driver / PDS	PcoDecoder (Vertex)
fragment_code	uint8_t[]	Driver / PDS	PcoDecoder (Fragment)
vertex_instructions	PcoInstruction[]	PcoDecoder	UscCluster (Vertex)
fragment_instructions	PcoInstruction[]	PcoDecoder	UscCluster (Fragment)
vertex_lanes	VertexLane[]	VertexFetch	UscCluster (Vertex), ClipCull
vertex_shared_registers	ShaderSharedRegister[]	Driver / PDS	UscCluster (Vertex)
usc_fragment_tasks	UscFragmentTask[]	PDS Engine	UscSlot, UscCluster
usc_coefficient_banks	uint32_t[]	PDS Engine	UscCluster (Fragment)
texture_sample_requests	TextureSampleRequest[]	UscCluster	TextureUnit

texture_sample_responses	TextureSampleResponse[]	TextureUnit	UscCluster
fragment_continuations	PcoFragmentContinuation[]	UscCluster	UscCluster (Resume)
fragment_outputs	FragmentOutput[]	UscCluster	PBE

8. Pseudocode 實作

8.1 PcoDecoder Pseudocode

```
FUNCTION PcoDecoder::Run():
    LOOP forever:
        txn = input_fifo.read()
        state = load(pool, txn.handle)

        code_handle = (stage == Vertex) ? state.vertex_code : state.fragment_code
        binary = load(pool, code_handle)

        // 1. 解碼為語意指令
        decoded = DecodePcoProgram(stage, binary)

        // 2. 介面契約檢查
        IF stage == Vertex:
            ASSERT (decoded.summary.vertex_input_mask & ~available_bindings) == 0
            ASSERT decoded.summary.vertex_output_mask == expected_linkage_mask
            ASSERT decoded.summary.ends_task == true
            state.vertex_program_summary = decoded.summary
            state.vertex_instructions = store(pool, decoded.instructions)
            state.vertex_groups = ceil(vs_invocations / 4)
            state.stage = kVertexDecoded
        ELSE:
            ASSERT decoded.summary.pixel_output_mask == 0x0F
            state.fragment_program_summary = decoded.summary
            state.fragment_instructions = store(pool, decoded.instructions)
            state.fragment_early_hsr_safe = decoded.summary.early_hsr_safe
            state.stage = kFragmentDecoded

        wait(decode_cycles)
        store(pool, txn.handle, state)
        output_fifo.write(txn)
```

8.2 UscSlot Pseudocode

```
FUNCTION UscSlot::Run():
    LOOP forever:
        txn = input_fifo.read()
        state = load(pool, txn.handle)

        IF stage == Vertex:
            ASSERT state.stage == kVertexDecoded
            lanes = state.counters.vs_invocations
            groups = state.vertex_groups
            ASSERT groups == ceil(lanes / 4)
            state.stage = kVertexIssued
        ELSE:
            ASSERT state.stage == kPdsReady
            quads = load(pool, state.fragment_quads)
            tasks = load(pool, state.usc_fragment_tasks)
            ASSERT quads.size() == tasks.size()
            groups = tasks.size()
            state.stage = kFragmentIssued

        cycles = usc_slot_base_cycles + ceil(groups / batch)
        wait(cycles)
        store(pool, txn.handle, state)
        output_fifo.write(txn)
```

8.3 UscCluster 頂點著色 Pseudocode

```
FUNCTION UscCluster::RunVertex():
    lanes[] = load(pool, state.vertex_lanes)
    insns[] = load(pool, state.vertex_instructions)
    shared[] = load(pool, state.vertex_shared_registers)

    FOR EACH lane IN lanes:
        // 初始化暫存器檔案 (VTXIN)
        vtxin = lane.vertex_input

        // 執行 PCO ISS
        execution = ExecuteVertexPco(state.vertex_program_summary, insns, vtxin, shared)

        // 寫出頂點屬性 (VTXOUT)
        lane.vertex_output = execution.outputs
        lane.emitted = execution.emitted
        lane.ended = execution.ended_task

    // 統計動態指令執行總數
    per_inv_counts = CountPcoInstructions(insns, expand_repeats=true)
    state.counters.vs_alu_instructions += per_inv_counts.alu * lanes.size()

    state.vertex_lanes = store(pool, lanes)
    state.stage = kVertexShaded
```

8.4 UscCluster 片段著色 (含紋理中斷與接續) Pseudocode

```
FUNCTION UscCluster::RunFragment():
    invocations[] = load(pool, state.fragment_invocations)
    insns[] = load(pool, state.fragment_instructions)

    IF !UsesTextureSampling(state):
        // --- 標準片段著色 (無紋理) ---
        outputs = []
        FOR EACH inv IN invocations:
            ctx.sample_x = inv.x
            ctx.sample_y = inv.y
            ctx.coefficients = GetCoefficients(inv.quad_id)
            execution = ExecuteFragmentPco(state.fragment_program_summary, insns, ctx)
            outputs.append(MakeFragmentOutput(inv, execution.pixel_outputs))

        state.fragment_outputs = store(pool, outputs)
        state.stage = kFragmentShaded

    ELSE:
        // --- 紋理片段著色 (非同步 SMP 流程) ---
        quads[] = load(pool, state.fragment_quads)
        shader_lanes[] = load(pool, state.fragment_shader_lanes)
        continuations = array(size=shader_lanes.size())
        requests = []

        // Phase 1: 執行至 SMP 暫停
        FOR EACH lane IN shader_lanes:
            ctx = SetupExecutionContext(lane)
            execution = ExecuteFragmentPco(state.fragment_program_summary, insns, ctx)
            ASSERT execution.suspended == 1

            continuations[lane.index] = execution.continuation
            requests.append(execution.texture_request)

        state.texture_sample_requests = store(pool, requests)
        state.fragment_continuations = store(pool, continuations)
        state.stage = kFragmentTexturePending
        texture_request_output.write(txn)

        // 等待 TPU 處理完成
        response_txn = texture_response_input.read()
        responses[] = load(pool, state.texture_sample_responses)

        // Phase 2: 接續執行與 WDF 喚醒
        outputs = []
```

```
FOR EACH lane IN shader_lanes:
    rgba = responses[lane.index].rgba
    saved_cont = continuations[lane.index]

    // 恢復執行後續指令
    execution = ResumeFragmentPco(state.fragment_program_summary, insns, saved_cont, rgba)
    ASSERT execution.written_mask == 0x0F

    IF !lane.is_helper:
        outputs.append(MakeFragmentOutput(lane, execution.pixel_outputs))

state.fragment_outputs = store(pool, outputs)
state.stage = kFragmentShaded
```